Employee Leave Management System (ELMS)

Streamlining Leave Approvals and Policy Queries with Microservices and AI

Submitted as Final Report, in fulfillment for the award of

TATA TECHNOLOGIES Pathway Internship Program

By:
Employee Name: Shreyash Jaiswal
Employee ID: 933511
LoB: CEIT

CERTIFICATE

This is to certify that project report titled **“Employee Leave Management System”**

Has been successfully completed and submitted by **Shreyash Jaiswal (Emp-ID – 933511)** under the guidance of **Mr. Kishore Mistry** at **Tata Technologies Ltd**.

The project report is Bonafide work carried out and submitted in partial fulfillment of the requirement for the compliment of the internship program.

Signature	Signature	Signature
Mr. Kishore Mistry	**Supriya Choudhary**	**Ms. Anuja Singh**
Tech Lead	**HRBP**	**Engagement Manager**

ACKNOWLEDGEMENTS

First and foremost, I wish to express our deepest gratitude to TATA Technologies, especially the dedicated team. Their unwavering support, both financially and in terms of infrastructure, played a pivotal role in the interim success of our project.

Special thanks are due to Kishore Mistry Sir, Tech Lead and Anuja Singh Ma'am, Engagement Manager. Their insights, expertise,and continuous encouragement have been invaluable. RM1 not only provided technical guidance but also readily stepped in whenever logistical challenges arose,ensuring the smooth progression of our project. Anuja Singh Maam's in-depth knowledge and constructive feedback have greatly enriched the development process, for which we are profoundly thankful.Additionally, we would like to acknowledge the team TechVarsity for operationally guiding us through our internship journey. Its availability and ease of access significantly facilitated our data collection and validation processes, proving instrumental in enhancing our software's efficacy.

By :
Employee Name: Shreyash Jaiswal
Employee ID : 933511
LoB : CEIT

TATA Technologies
Blue Ridge Town Pune, Phase 1, Hinjawadi Rajiv Gandhi Infotech ParkHinjawadi,
Pimpri-ChinchwadMaharashtra, 411057

TABLE OF CONTENTS

1. Table of Contents.....Pg. 03

2. List of Tables.....Pg. 04

3. List of Figures.....Pg. 05

4. List of Abbreviations.....Pg. 06

5. List of Symbols.....Pg. 07

6. Abstract.....Pg. 08

7. Introduction & Problem Statement.....Pg. 09

8. Industry Overview.....Pg. 10

9. Work Responsibility during Internship.....Pg. 11

10. Literature Survey.....Pg. 12

11. Design Methodology.....Pg. 14

12. Results.....Pg. 19

13. Conclusion.....Pg. 28

14. Summary.....Pg. 29

15. References *(Technical Journals / website the interns have referenced in the report)* .....Pg. 30

16. Bibliography *(study material read to gain background knowledge)* .....Pg. 31

17. Annexure – Interim Report.....Pg. 32

2. LIST OF TABLES

Table No.	Table Title	Page No.
10.1	Existing Leave Management System	Pg. 12
10.2	Technology Stack and Justification	Pg. 12
11.3.1	Table auth_users	Pg. 16
11.3.2	Table user_profiles	Pg. 16
11.3.3	Table leave_requests	Pg. 16
11.3.4	Table leave_balances	Pg. 16
11.4	Role-Based Access Control Matrix	Pg. 17
11.5	Application Routing and Navigation	Pg. 17
12.1	Functional Test Results	Pg. 19

3. LIST OF FIGURES

Figure No.	Figure Title	Page No.
11.2	High Level System Architecture Design	Pg. 14
11.3	Database Entity Relationship Diagram	Pg. 15
11.6	Component Architecture	Pg. 17
12.2.1	Login Page	Pg. 20
12.2.2	Register Page	Pg. 20
12.2.3	Employee's Dashboard	Pg. 21
12.2.4	Apply Leave	Pg. 21
12.2.5	My Leaves	Pg. 22
12.2.6	Employee's Leave Balance	Pg. 22
12.2.7	ChatBot	Pg. 23
12.2.8	ChatBot Response	Pg. 23
12.2.9	Manager's Dashboard	Pg. 24
12.2.10	My Team	Pg. 24
12.2.11	Pending Leaves	Pg. 25
12.2.12	Admin Dashboard	Pg. 25
12.2.13	Initialize Balance	Pg. 26
12.2.14	Employee Directory	Pg. 26
12.2.15	Admin's Chatbot PDF Upload and Response	Pg. 27

4. LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
AI	Artificial Intelligence
CRUD	Create, Read, Update, Delete
ELMS	Employee Leave Management System
HRMS	Human Resource Management System
JSON	JavaScript Object Notation
JWT	JSON Web Token
RDBMS	Relational Database Management System
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
UI/UX	User Interface / User Experience

5. LIST OF SYMBOLS

Symbol	Description
→	Indicates state transition in leave lifecycle or microservice communication
✓	Test case passed
✗	Test case failed

6. ABSTRACT

Managing employee leaves, tracking balances, and navigating complex HR policies are critical but often cumbersome processes for large organizations. Traditional methods involving email chains, spreadsheets, or monolithic legacy systems lead to delays in approvals, confusion regarding policy rules, and a lack of transparency for employees and managers alike.

This project presents the design and development of the **Employee Leave Management System (ELMS)**, a highly scalable, web-based platform built on a modern Microservices architecture. The system enables employees to manage leave balances and apply for time off, allows managers to review department-specific requests, and provides administrators with comprehensive organizational oversight.

The application is architected with a **React.js Single Page Application (SPA)** on the frontend and a suite of **Spring Boot Microservices** (User Service, Leave Service, Auth Service, API Gateway, and Eureka Discovery Server) on the backend. Data is reliably stored using a relational **MySQL** database, while secure identity management is handled via **JWT (JSON Web Tokens)**. Role-Based Access Control (RBAC) ensures distinct, secure experiences for Users (Employees), Managers, and Administrators.

A standout feature of ELMS is the **AI-powered Chatbot**, integrated via the NVIDIA NIM API. This intelligent assistant uses Retrieval-Augmented Generation (RAG) to parse uploaded HR PDF policies and securely accesses real-time database context (e.g., pending leaves, team directories) through backend services. This allows employees to ask complex policy questions or check their status conversationally.
The system was developed iteratively, tested for functional integration, and features a modern, modern, light-themed UI utilizing animated gradient backgrounds and glassmorphism aesthetics. Results demonstrate a highly responsive, scalable, and intelligent HR tool capable of modernizing the leave management lifecycle.

Keywords: Leave Management System, Microservices, Spring Boot, React.js, MySQL, AI Chatbot, Generative AI, RAG, API Gateway, JWT Authentication.

7. INTRODUCTION & PROBLEM STATEMENT

7.1 Introduction

In modern enterprises, workforce management—specifically the tracking and approval of employee time off—demands a streamlined, transparent, and accurate system. Tata Technologies, with its extensive global workforce, requires internal systems that not only record data but actively assist employees and managers in adhering to company policies.

Human Resource Management Systems (HRMS) are evolving from static data repositories to intelligent platforms. A modern leave management system must provide real-time balances, enforce role-based hierarchical approvals, and offer immediate answers to policy-related queries without burdening the HR department. By adopting a microservices architecture, organizations can achieve the high availability, fault tolerance, and independent scalability required for enterprise applications.

This project focuses on building ELMS (Employee Leave Management System) from the ground up, moving away from monolithic designs to a distributed Spring Boot backend, coupled with an AI-driven assistant to elevate the employee experience.

7.2 Problem Statement

The current processes for leave management face the following challenges:

1. **Lack of Centralized, Real-Time Tracking:** Leave requests managed via email or disparate tools lack a unified dashboard, leading to miscalculated leave balances and scheduling conflicts.
2. **Policy Confusion and HR Overhead:** Employees frequently have questions regarding leave policies (e.g., "Can I carry forward casual leave?"). Finding answers in lengthy PDF manuals is tedious, resulting in repetitive queries to HR.
3. **Monolithic Architecture Limitations:** Legacy systems are difficult to scale or update. If the reporting module fails, the entire application goes down.
4. **Poor Managerial Visibility:** Managers lack department-scoped dashboards that easily group their team's pending requests, slowing down the approval pipeline.

7.3 Objectives

The primary objectives of this project are:

- To design a highly scalable, distributed backend using Spring Boot microservices, Eureka Server, and an API Gateway.
- To implement Role-Based Access Control (RBAC) tailored for Employees, Managers, and Admins using secure JWT authentication.
- To develop a dynamic React.js frontend with intuitive dashboards and cascading department-based views.
- To integrate an intelligent AI Chatbot capable of reading HR policy PDFs and interacting with real-time user database context to answer queries accurately.
- To ensure persistent, reliable data storage using a relational MySQL database.

7.4 Scope

The scope of this project encompasses:

- Development of a React.js-based SPA with modern UI/UX principles.
- Development of independent Spring Boot microservices (Auth, User, Leave, AI).
- Integration with MySQL for relational data management (User profiles, Leave requests, Balances).
- Implementation of an AI chat interface communicating with NVIDIA NIM API.
- Out of scope: Payroll integration and advanced multi-level HR escalations (identified as future enhancements).

8. INDUSTRY OVERVIEW

8.1 HR Tech and Human Capital Management (HCM) Industry

The global Human Resource Technology and Human Capital Management (HCM) market has experienced unprecedented growth, driven by digital transformation initiatives and the global shift toward hybrid work environments. According to industry reports, the core HR software market was valued at approximately USD 24 billion in 2023 and is projected to grow at a Compound Annual Growth Rate (CAGR) of over 10% through 2030. This growth is heavily fueled by organizations seeking to replace legacy, paper-based processes with automated, cloud-based workflow orchestration tools that enhance employee experience and reduce administrative bottlenecks.

8.2 Enterprise HR Systems Landscape

The enterprise HR software landscape is dominated by comprehensive commercial solutions such as Workday, SAP SuccessFactors, BambooHR, and Oracle HCM. These platforms offer end-to-end capabilities spanning recruitment, payroll, performance management, and leave tracking. However, they frequently entail substantial licensing costs, rigid implementation timelines, and complex feature sets that far exceed the specific, lightweight requirements of internal leave management use cases. Organizations often find that forcing simple internal processes into heavy, vendor-locked ecosystems reduces agility and increases operational overhead.

8.3 Tata Technologies Context

Tata Technologies Limited is a premier global product engineering and digital services company serving the automotive, aerospace, and industrial heavy machinery sectors. With a workforce of over 12,500 employees distributed across 27 countries, the organization demands highly efficient, customized internal service management systems to support its operations. The Employee Leave Management System (ELMS) project was specifically conceptualized and developed to provide a lightweight, purpose-built, and cost-effective alternative to commercial HRMS platforms, tailored exactly to the organizational structure, departmental hierarchy, and specific HR functions of the company.

8.4 Technology Trends

Key technology trends influencing the architecture and design of this project include:

- Microservices Architecture: The industry is rapidly shifting away from monolithic backends toward distributed, independently deployable microservices (e.g., using Spring Boot and Netflix Eureka). This ensures high fault tolerance, isolated scaling, and easier maintenance.
- Component-Based Frontend Architecture: Modern frameworks like React.js enable modular, maintainable User Interface (UI) development through reusable components, declarative rendering, and efficient state management without full page reloads.
- Generative AI & RAG (Retrieval-Augmented Generation): Enterprise applications are increasingly integrating Large Language Models (LLMs) to provide intelligent conversational interfaces. RAG allows these models to securely reference proprietary internal data (like HR PDFs and real-time database context) to provide highly accurate, hallucination-free assistance.
- Modern UI/UX Principles: Contemporary enterprise applications are adopting bright, clean aesthetics with dynamic animated gradient backgrounds, glassmorphism aesthetics, and responsive micro-animations to reduce visual fatigue, improve user engagement, and deliver a consumer-grade experience in workplace tools.

9. WORK RESPONSIBILITY – INTERNSHIP PERIOD

9.1 Overview of Responsibilities

During the internship period at Tata Technologies, the following responsibilities were undertaken:

Phase	Duration	Activities
Phase 1: Research & Planning	Week 1	Requirements gathering, technology stack selection (Spring Boot, React, MySQL), architectural design of microservices, and setting up the development environment.
Phase 2: Core Backend Microservices	Week 1-2	Developed auth-service (JWT generation), user-service (Profile management, MySQL integration), and leave-service (Leave logic, balances, status tracking). Configured Eureka Server and Spring Cloud API Gateway for routing.
Phase 3: Frontend Development	Week 3	Built React components: Authentication flows, Employee Dashboard, Manager "My Team" view with cascading department logic, and Admin Directory.
Phase 4: AI & Advanced Integration	Week 4	Developed ai-service in Spring Boot. Integrated PDF parsing and real-time context fetching (inter-service communication via RestTemplate). Connected frontend chat UI to Nvidia NIM API for intelligent responses.
Phase 5: Testing & Deployment	Week 4	End-to-end integration testing, resolving CORS and gateway routing issues, UI polishing (light theme, animated backgrounds, glassmorphism), and documenting the final report.

9.2 Key Deliverables

1. Fully functional ELMS web application with a React.js frontend.
2. Distributed backend architecture comprising 5 Spring Boot applications.
3. Relational MySQL database schemas for secure data persistence.
4. Role-based dashboards featuring department-scoped data viewing for Managers.
5. Context-aware AI Chatbot capable of reading HR policies and user data.

10. LITERATURE SURVEY

10.1 Existing Leave Mangement Systems

Feature	Workday / SAP SuccessFactors	BambooHR	ELMS (This Project)
Deployment	Cloud/On-Prem	Cloud	Cloud-Ready Microservices
Cost	High (Enterprise)	Medium (SMB)	Custom/In-house
Architecture	Monolithic / SOA	Monolithic	Microservices
AI Integration	Add-on modules	Basic chatbots	Deep contextual RAG AI
Customization	Complex	Moderate	Fully Custom (Code-level)
Database	Proprietary	Relational	MySQL

10.2 Technology Stack and Justification

Component	Technology	Justification
Frontend	React.js	Component-based architecture, efficient virtual DOM, wide community support.
Backend Framework	Spring Boot (Java)	Enterprise-grade, rapid development, robust security, and native microservices support.
Service Discovery	Netflix Eureka	Dynamic service registration and discovery, eliminating hardcoded IPs.
API Gateway	Spring Cloud Gateway	Centralized routing, single entry point, and global security filtering.
Database	MySQL	Relational integrity, ACID compliance, excellent for structured transactional data (users, leaves).
Authentication	JWT (JSON Web Tokens)	Stateless, scalable, secure token-based authentication across microservices.
AI Integration	NVIDIA NIM API	High-performance, secure inference for Large Language Models.

10.3 React.js as a Frontend Framework

React.js, developed and maintained by Meta, is an open-source JavaScript library for building user interfaces based on a component-based architecture. Key advantages include:

- **Virtual DOM:** React maintains a virtual representation of the DOM, enabling efficient reconciliation and minimizing costly direct DOM manipulations.
- **Component Reusability:** Encapsulated components with defined props and state enable modular development and code reuse across the dashboards.
- **Declarative Rendering:** UI state is expressed declaratively, making the code more predictable and easier to debug during state transitions.

10.4 Spring Boot and Microservices Architecture

Spring Boot is an extension of the Spring framework that simplifies the setup and development of robust, enterprise-grade Java applications. By utilizing a Microservices architecture, the backend is divided into loosely coupled services (auth-service, user-service, leave-service, ai-service). Key advantages include:

- **Fault Tolerance & Isolation:** If the AI module crashes, core leave approval functionality remains fully operational.
- **Service Discovery (Eureka):** Netflix Eureka allows services to find each other dynamically without hardcoded IP addresses, enabling seamless scaling.
- **API Gateway:** Spring Cloud Gateway acts as a single-entry point, handling routing, Cross-Origin Resource Sharing (CORS), and centralized JWT token validation.

10.5 MySQL as a Relational Database

MySQL is a widely adopted open-source Relational Database Management System (RDBMS). It was selected for this project because HR and leave management data (users, balances, request states) are highly structured and require strict ACID (Atomicity, Consistency, Isolation, Durability) compliance to prevent anomalies like over-deducting leave balances.

10.6 Role-Based Access Control (RBAC) & JWT Security

RBAC is a security paradigm that restricts system access based on assigned roles. In ELMS, this ensures:

- **Least Privilege:** Employees only access their own data, Managers access their department's data, and Admins have global read/write access.
- **Stateless Security via JWT:** JSON Web Tokens are generated by the auth-service upon login. The API Gateway validates these tokens on every subsequent request without needing to query a session database, vastly improving backend performance.

10.7 NVIDIA NIM & Generative AI (RAG)

The system integrates the NVIDIA NIM API to provide an intelligent conversational interface. Using the Retrieval-Augmented Generation (RAG) pattern, the system retrieves real-time JSON data from the database and extracted text from HR policy PDFs, injecting it directly into the LLM's system prompt. This ensures the chatbot provides highly accurate, hallucination-free answers based strictly on organizational data.

10.8 Modern UI/UX Design Principles

Contemporary web application design emphasizes:

- **Dynamic Backgrounds & Light Theme:** Utilizes animated gradient bars and a clean white aesthetic to create a vibrant, engaging interface that feels significantly more modern than legacy enterprise tools.
- **Glassmorphism:** A design trend featuring frosted-glass effects achieved through background blur and transparency, creating depth and visual hierarchy.
- **Responsive Design:** Ensuring consistent functionality and aesthetics across desktop and tablet interfaces.

11. DESIGN METHODOLOGY

11.1 Software Development Methodology

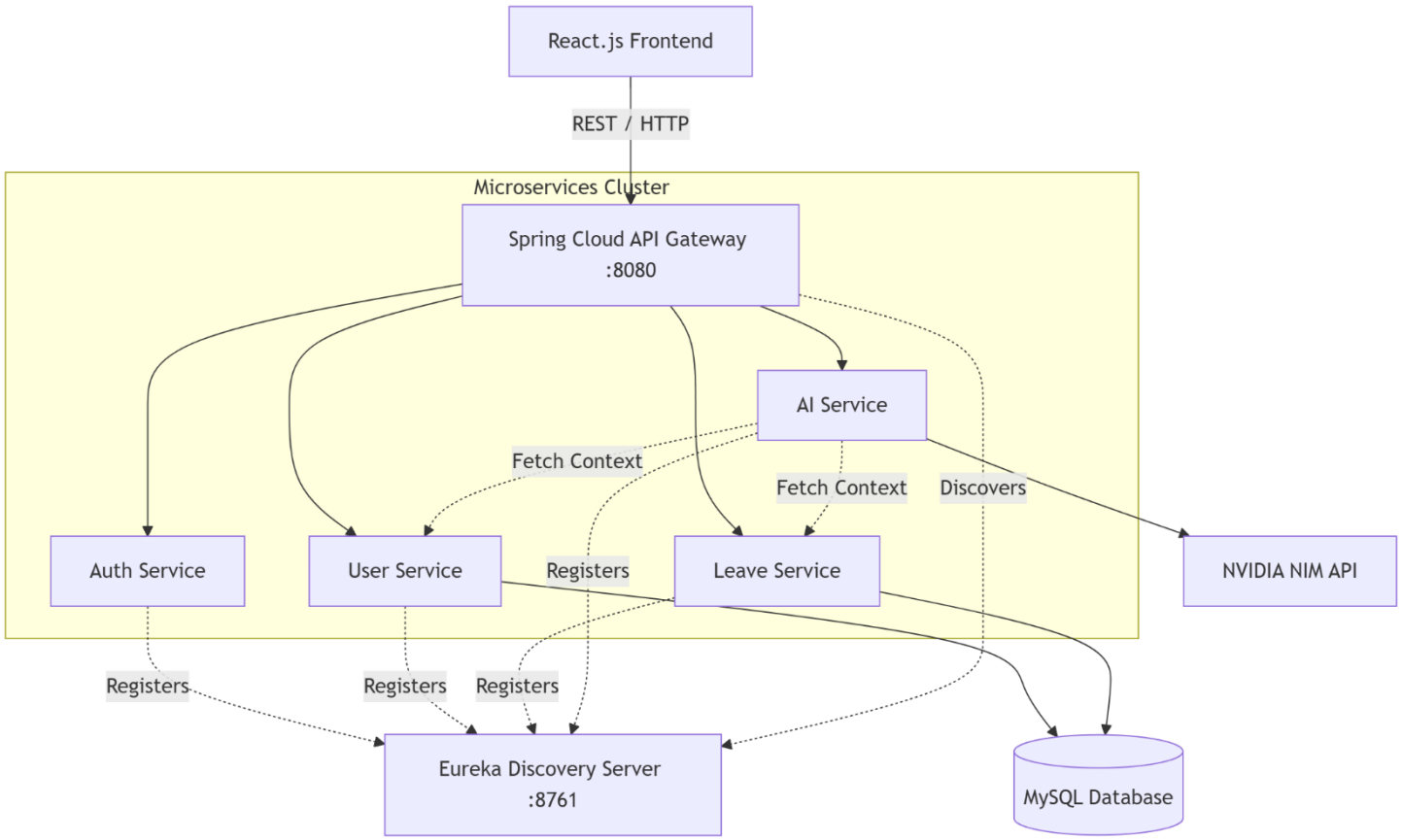
The project followed an **Agile development methodology**. Development was broken down into iterative sprints, starting with core database design, moving to individual microservice creation, followed by frontend consumption, and culminating in advanced AI integration.

11.2 System Architecture

ELMS utilizes a **Microservices Architecture**.

1. **Frontend (React.js):** Communicates exclusively with the API Gateway.
2. **API Gateway (Port 8080):** Routes requests (/auth/**, /users/**, /leaves/**, /ai/**) to the appropriate backend service. Validates JWT tokens for secured routes.
3. **Eureka Server (Port 8761):** Acts as a registry. All microservices register themselves here on startup.
4. **Microservices:**
    - auth-service: Handles login/registration and issues JWTs.
    - user-service: Manages employee profiles and department structures (MySQL).
    - leave-service: Handles leave applications, balance deductions, and manager approvals (MySQL).
    - ai-service: Aggregates data from leave-service and user-service, parses PDFs, and constructs prompts for the NVIDIA LLM.

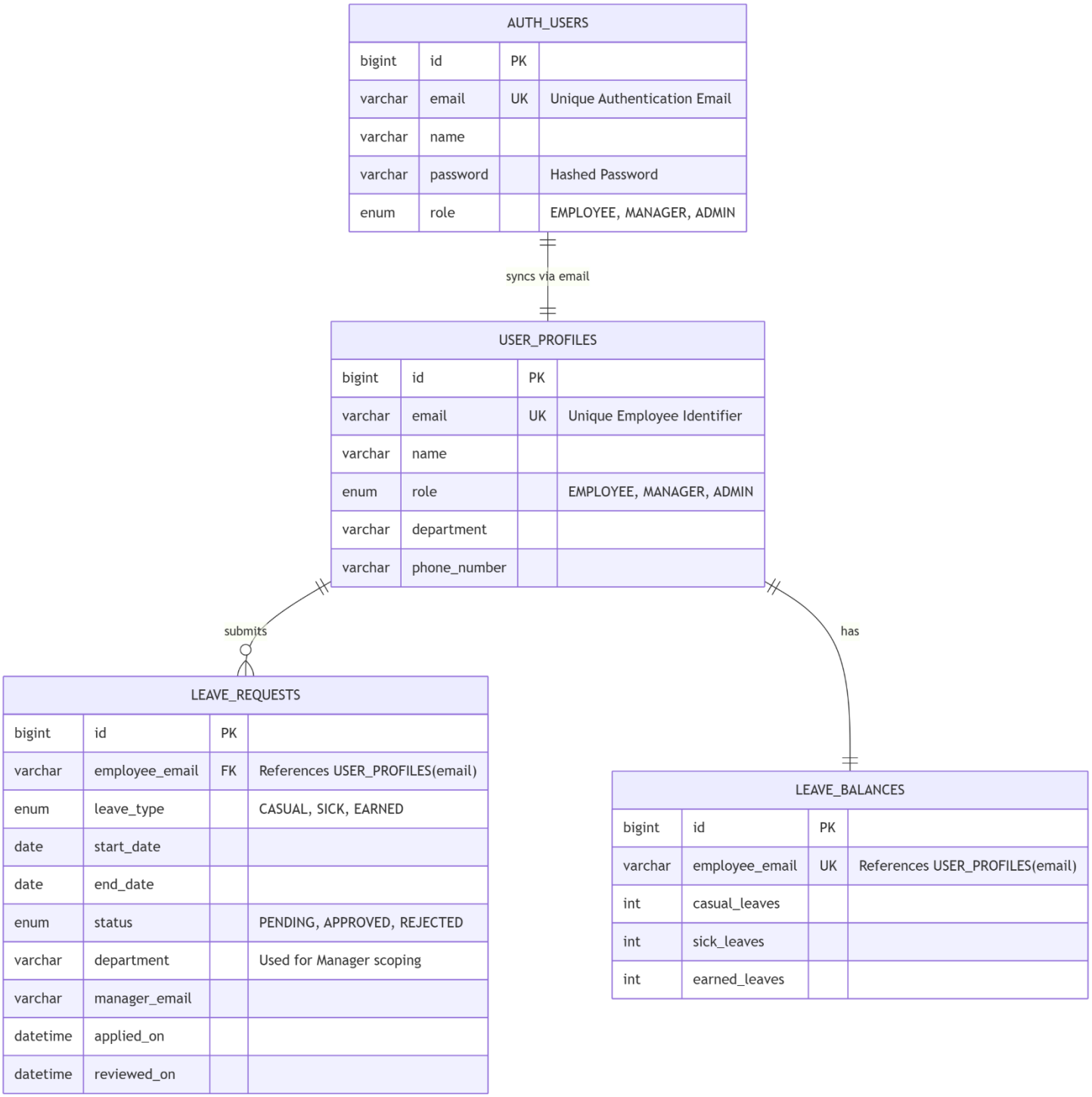
Figure 11.2 High Level System Design Architecture



11.3 Database Design (MySQL Schema)

The database schema is heavily normalized to ensure data integrity across the user-service and leave-service.

Figure 11.3 Data base ER-Diagram



11.3.1 Table auth_users (in auth_db)

Field	Type	Description
id	Big Int (PK)	Auto-generated unique credential identifier
email	Varchar (UK)	Employee email address used for login
name	Varchar	Employee full name
password	Varchar	Securely hashed password
role	Enum	"EMPLOYEE", "MANAGER", or "ADMIN"

11.3.2 Table user_profiles (in user_db)

Field	Type	Description
id	Big Int (PK)	Auto-generated unique user identifier
email	Varchar (UK)	Employee email address (Used for auth/identification)
name	Varchar	Employee full name
role	Enum	"EMPLOYEE", "MANAGER", or "ADMIN"
department	Varchar	Department or LoB (Used for manager scoping)
phone_number	Varchar	10-digit contact number

11.3.3 Table leave_requests

Field	Type	Description
id	Big Int (PK)	Auto-generated unique leave request identifier
employee_email	Varchar (FK)	Email of the person raising the request
leave_type	Enum	"CASUAL", "SICK", or "EARNED"
start_date	Date	Start date of the leave
end_date	Date	End date of the leave
status	Enum	"PENDING", "APPROVED", or "REJECTED"
department	Varchar	Department of the employee (used for routing to manager)
manager_email	Varchar	Email of the manager who reviewed the request
applied_on	Datetime	Server-side creation timestamp
reviewed_on	Datetime	Resolution timestamp (null until reviewed)

11.3.4 Table leave_balances

Field	Type	Description
id	Big Int (PK)	Auto-generated unique record identifier
employee_email	Varchar (UK)	Email of the employee
casual_leaves	Int	Remaining casual leave balance
sick_leaves	Int	Remaining sick leave balance
earned_leaves	Int	Remaining earned leave balance

11.4 Role-Based Access Control (RBAC) Matrix

Capability	Employee	Manager	Admin
Apply for Leave	✓	✓	✗
View Own Leaves & Balance	✓	✓	✗
View Pending Leaves	✗	✓ (Department only)	✓ (All)
Approve/Reject Leaves	✗	✓ (Department only)	✓ (All)
View Employee Directory	✗	✓ (Department only)	✓ (All)
AI Chatbot Access	✓ (Personal context)	✓ (Team context)	✓ (Global context)

11.5 Application Routing and Navigation

Route	Component	Access
/	Home (Landing Page)	Public
/login	Login	Public
/register	Register	Public
/employee-dashboard	EmployeeDashboard	Authenticated (role: EMPLOYEE)
/manager-dashboard	ManagerDashboard	Authenticated (role: MANAGER)
/admin-dashboard	AdminDashboard	Authenticated (role: ADMIN)
/apply-leave	LeaveApplicationForm	Authenticated (role: EMPLOYEE, MANAGER)

11.6 Component Architecture

Figure 11.6 Component Architecture

```
App.js
├── Home.js                    (Landing page)
├── Login.js                   (Authentication - Login)
├── Register.js                (Authentication - Registration)
├── EmployeeDashboard.js       (Employee role - View balance & apply leave)
│   └── LeaveApplicationForm.js(Form to submit new leave request)
├── ManagerDashboard.js        (Manager role - View team pending approvals)
├── AdminDashboard.js          (Admin role - Global metrics and directory)
├── ChatbotWidget.js           (Floating AI chat interface)
└── Navbar.js                  (Navigation bar component)
```

11.7 Authentication Flow

1. **Registration:** User enters Name, Email, Password, Department, and Role. The auth-service securely hashes the password and stores credentials in auth_db. Simultaneously, the user-service creates a corresponding business profile in user_db.
2. **Login:** User enters Email and Password. The auth-service validates credentials and generates a secure JSON Web Token (JWT) signed with a private secret key.
3. **Session Management:** The JWT is stored in the browser's localStorage. For every subsequent API call (e.g., fetching leave balances), the frontend attaches the JWT as a Bearer token in the HTTP Authorization header.
4. **Gateway Validation:** The Spring Cloud API Gateway intercepts the request, verifies the JWT signature, and routes the authenticated request to the appropriate microservice.
5. **Logout:** The JWT is cleared from localStorage and the user is redirected to the login page.

12. RESULTS

12.1 Functional Test Results

t ID	Test Case	Expected Output	Actual Output	Status
TC-01	User registration	Profile created in user-service, credentials in auth-service	Created successfully	✓ Pass
TC-02	Login with valid credentials	API Gateway returns valid JWT token	Token received	✓ Pass
TC-03	Apply for leave (sufficient balance)	Leave created as PENDING, balance deducted	Stored in MySQL	✓ Pass
TC-04	Apply for leave (insufficient balance)	System blocks request, returns HTTP 400 Bad Request	Error displayed	✓ Pass
TC-05	Manager views pending queue	API returns only leaves matching Manager's department	Filtered correctly	✓ Pass
TC-06	Inter-service communication	leave-service successfully fetches department from user-service via RestTemplate	Data retrieved	✓ Pass
TC-07	Chatbot: Policy query	AI reads PDF context and answers correctly	Accurate response	✓ Pass
TC-08	Chatbot: Context query (Manager)	AI successfully lists team members by querying user-service	Accurate response	✓ Pass

12.2 Dashboard Screenshots

Figure 12.2.1 Login Page :

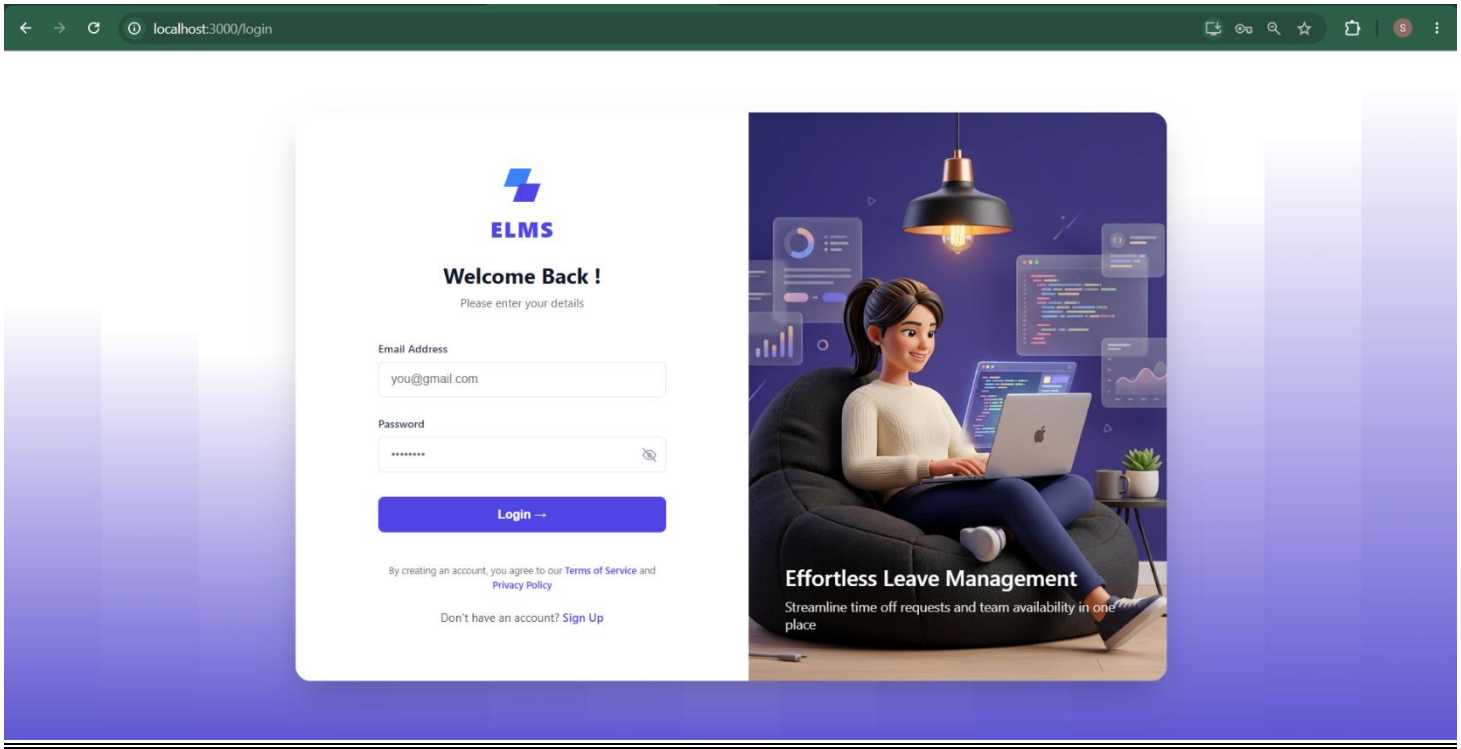


Figure 12.2.2 Register Page:

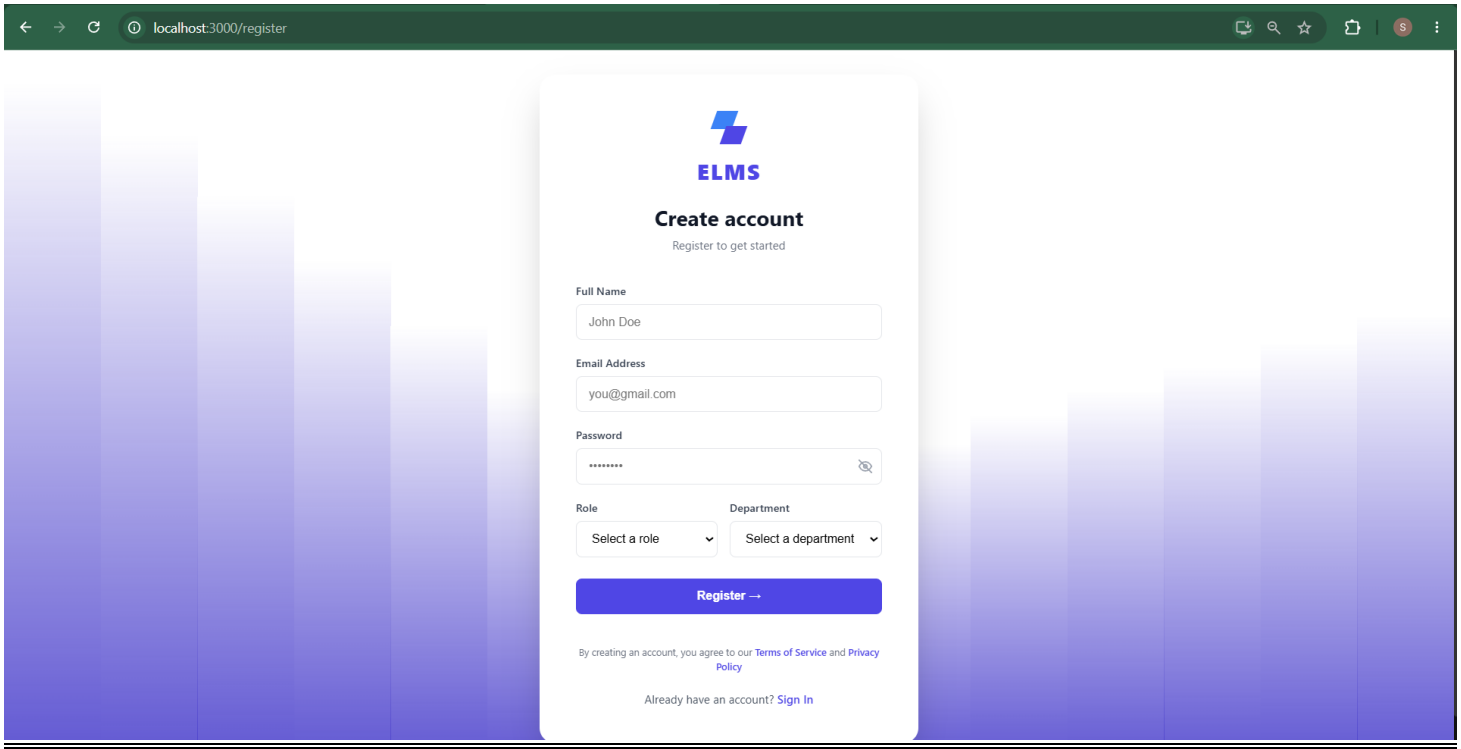


Figure 12.2.3 Employee's Dashboard :

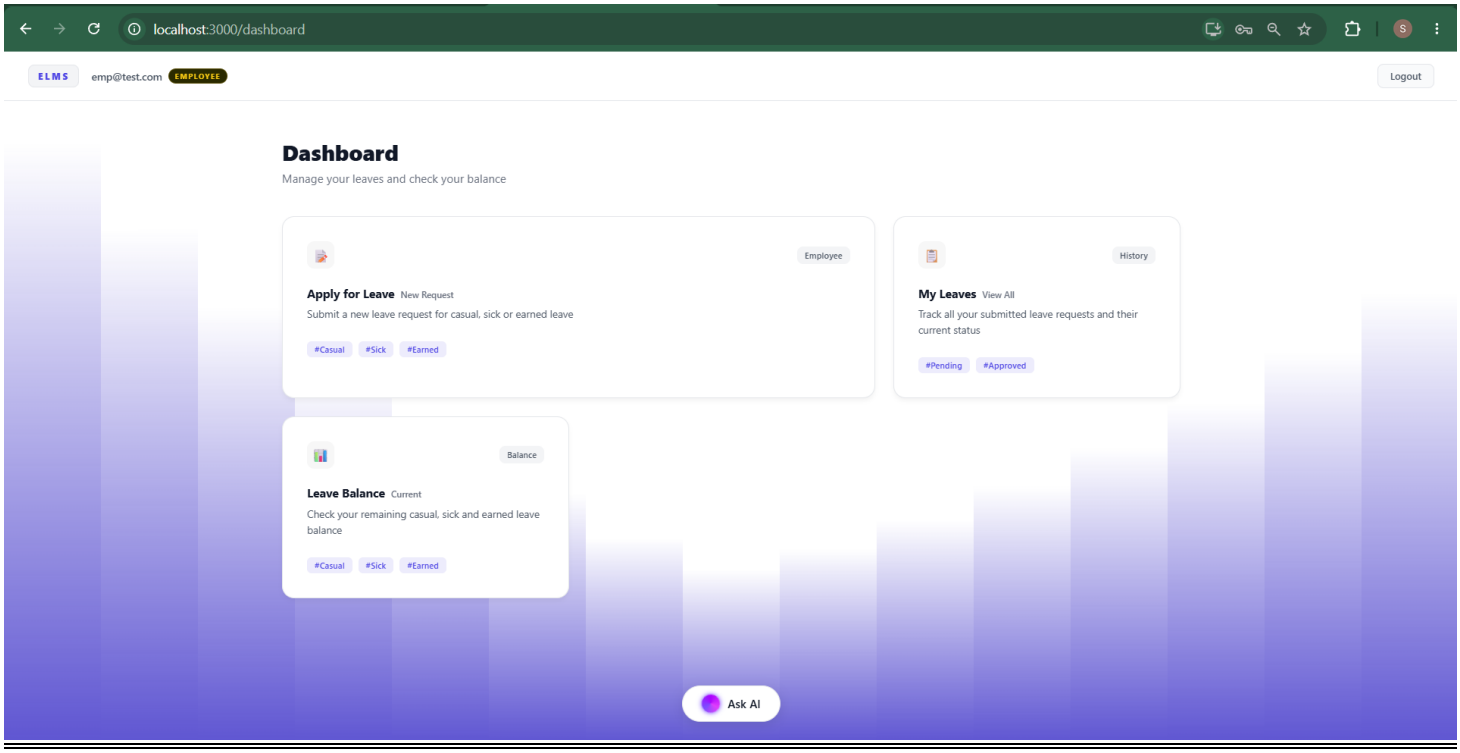


Figure 12.2.4 Apply Leave:

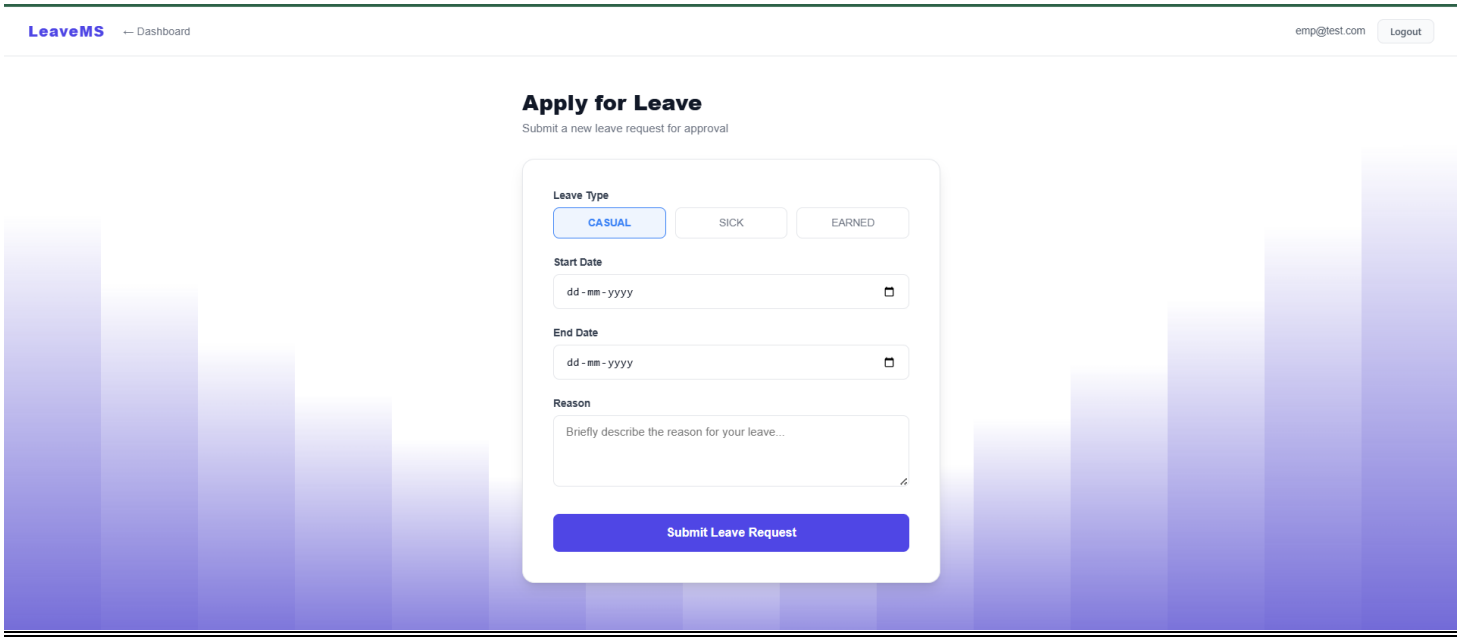


Figure 12.2.5 My Leaves:

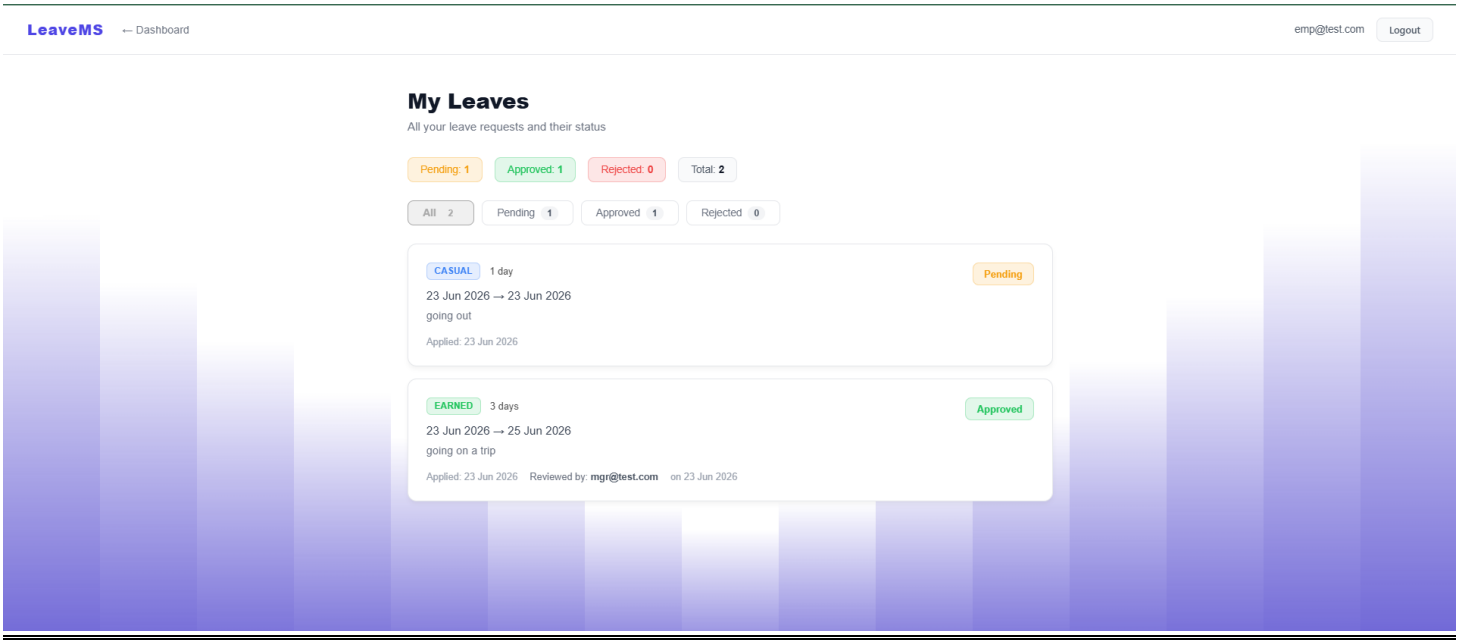


Figure 12.2.6 Employee's Leave Balance :

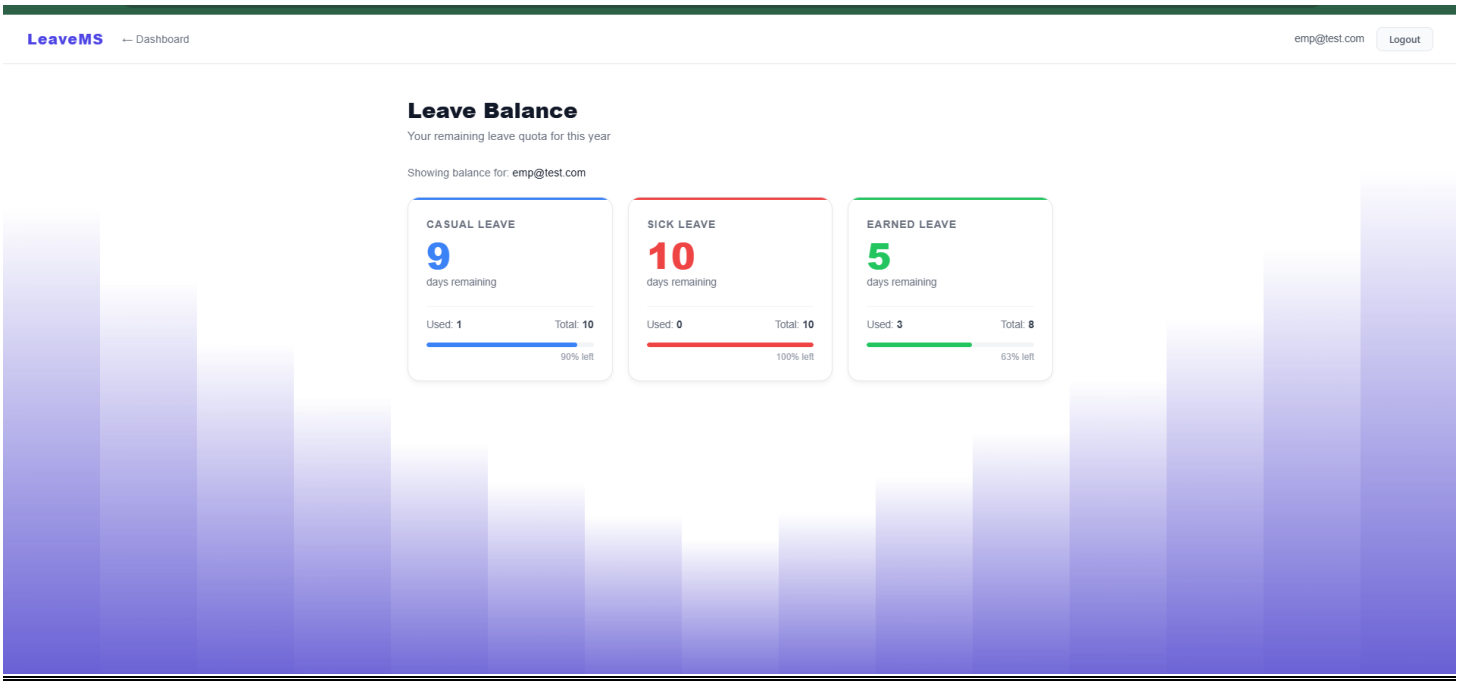


Figure 12.2.7 ChatBot:

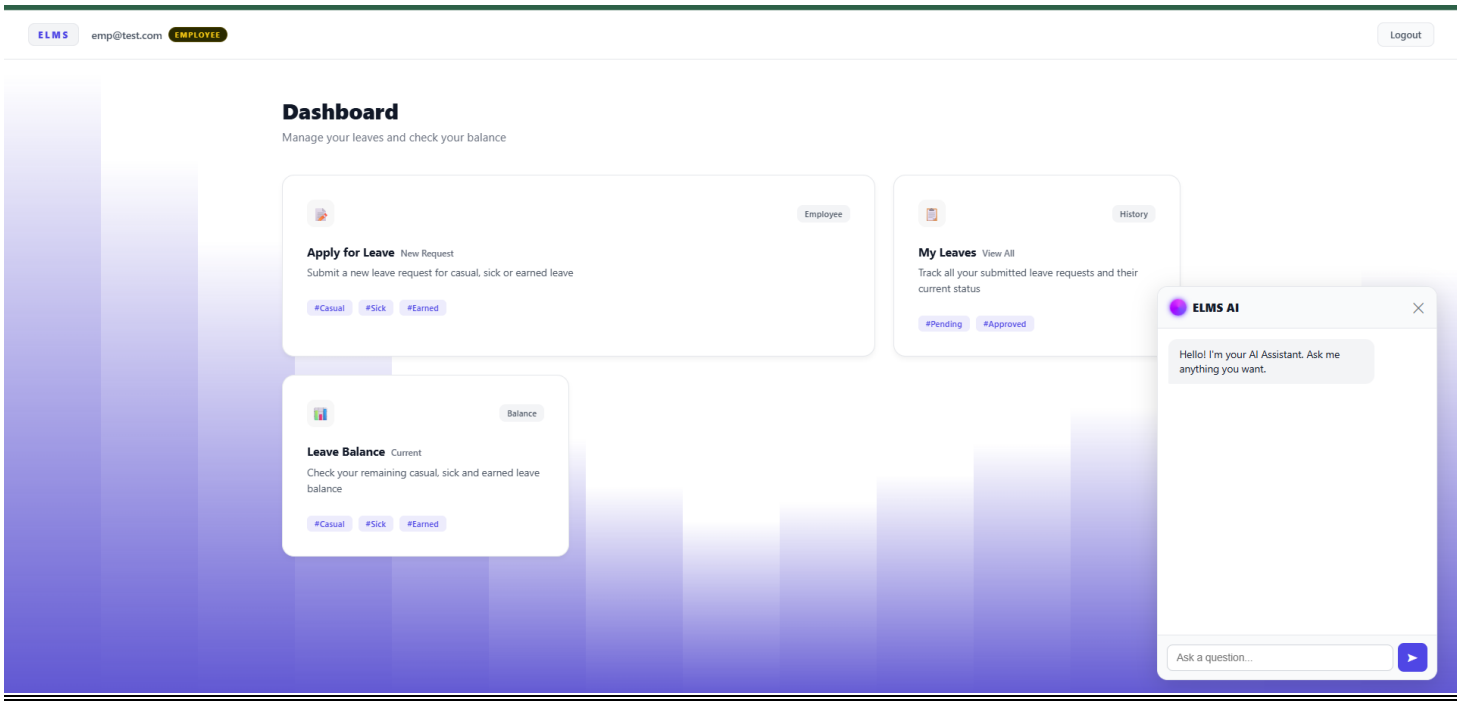


Figure 12.2.8 ChatBot Response:

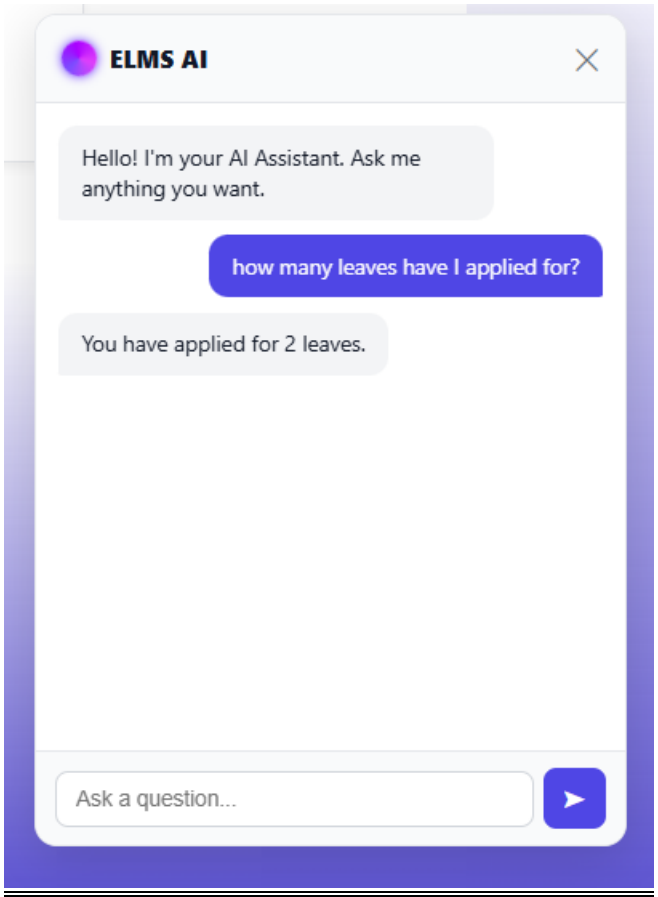


Figure 12.2.9 Manager's Dashboard:

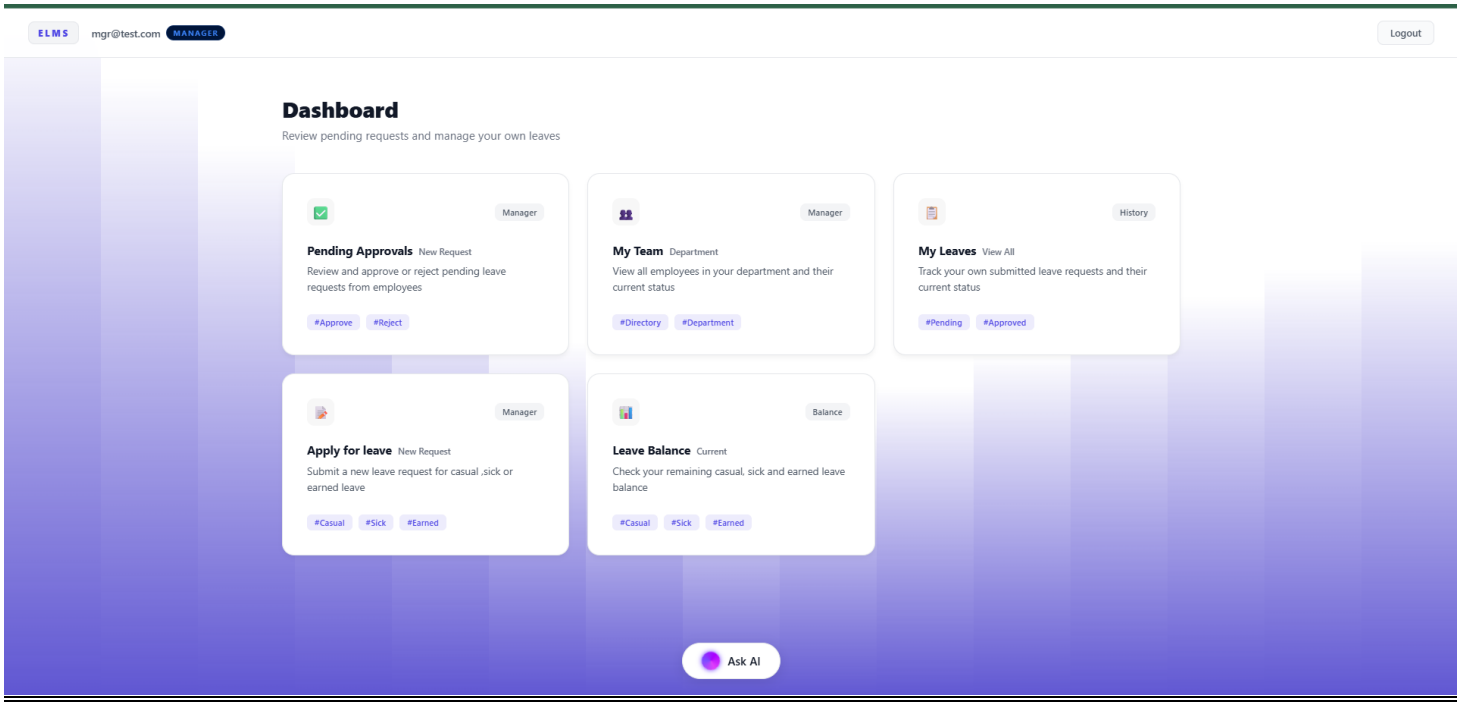


Figure 12.2.10 My Team:

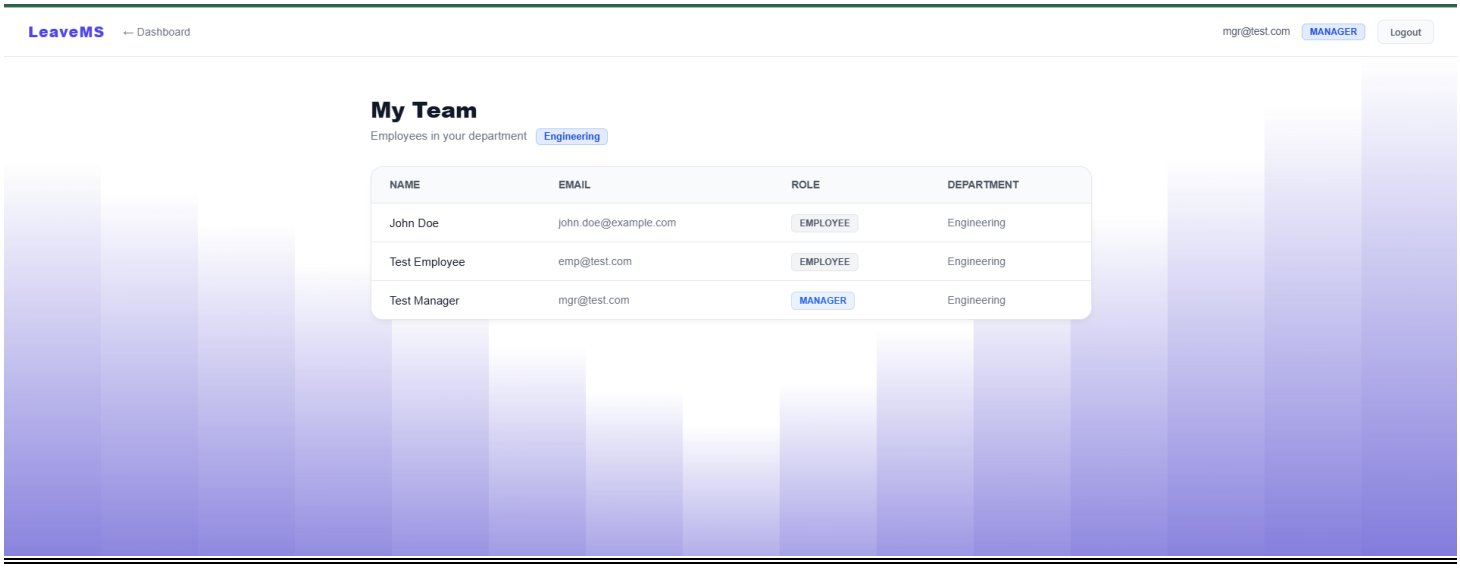


Figure 12.2.11 Pending Approvals:

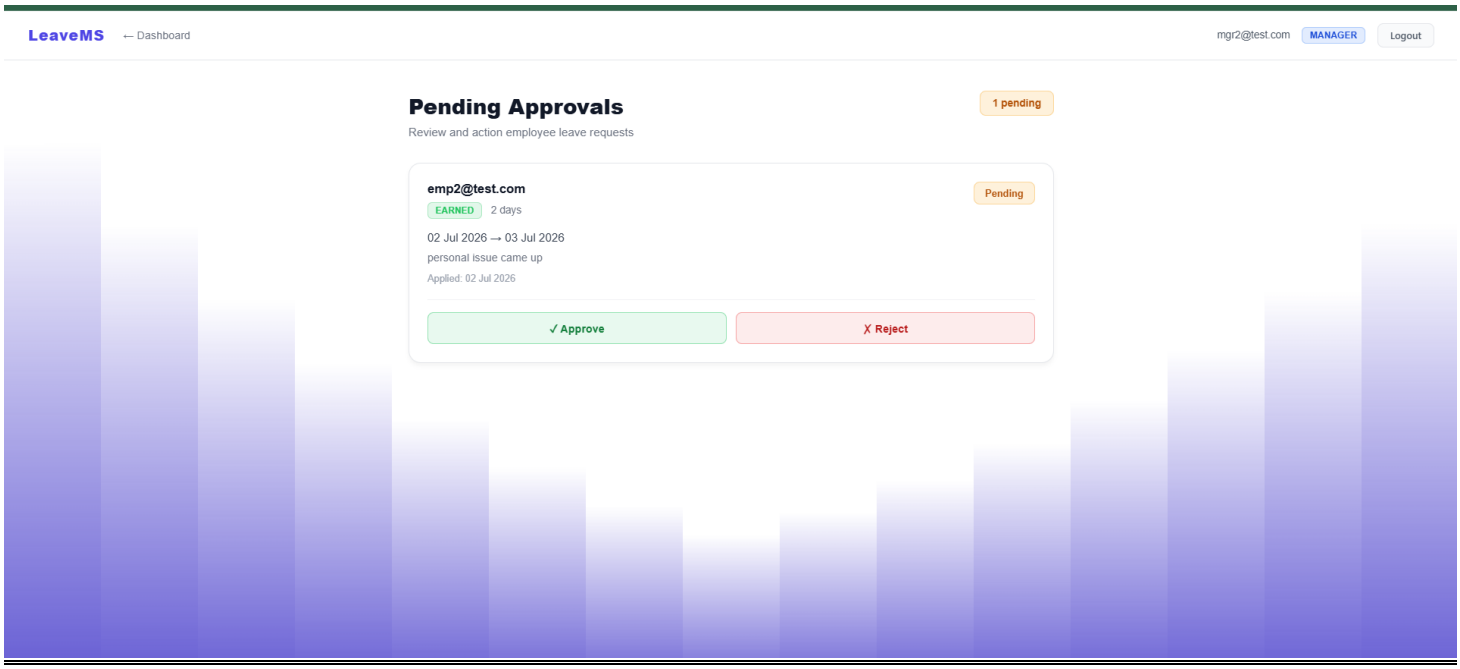


Figure 12.2.12 Admin Dashboard:

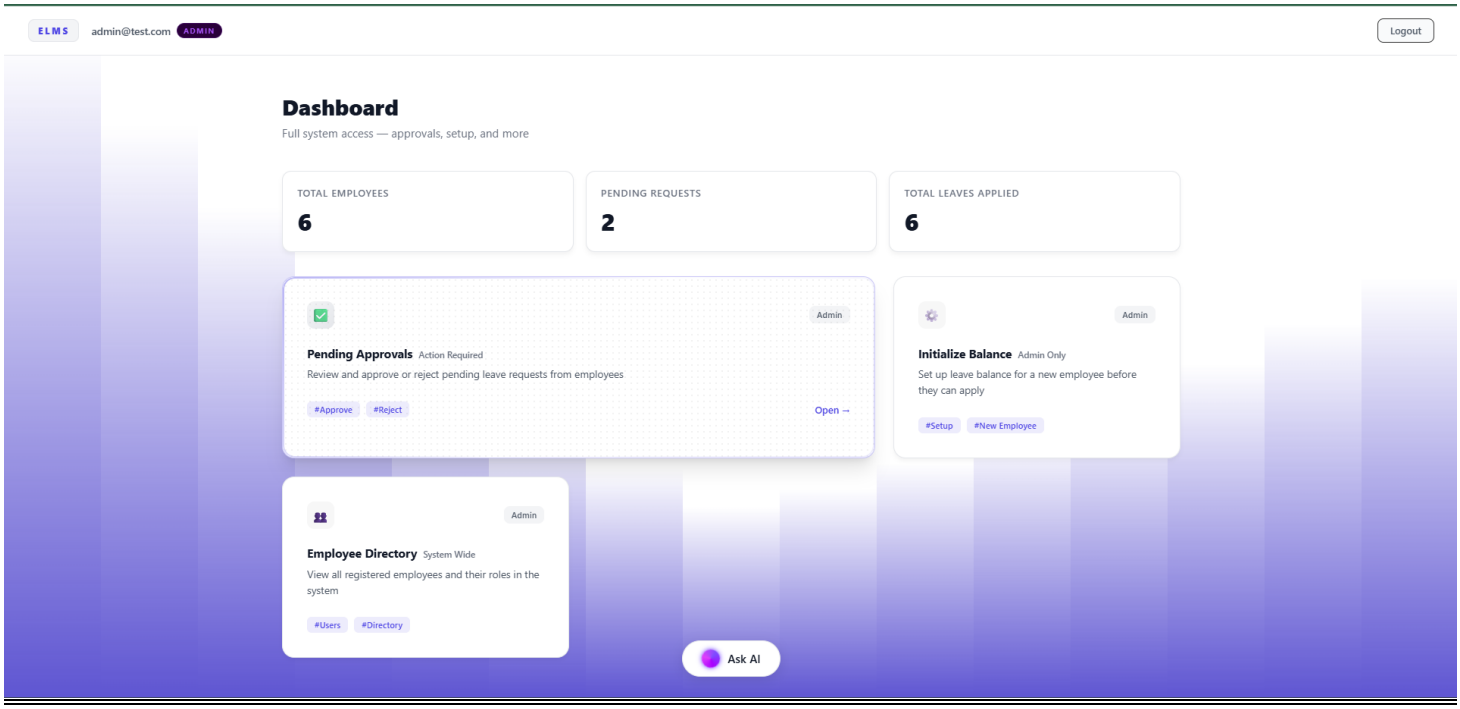


Figure 12.2.13 Initialize Balance:

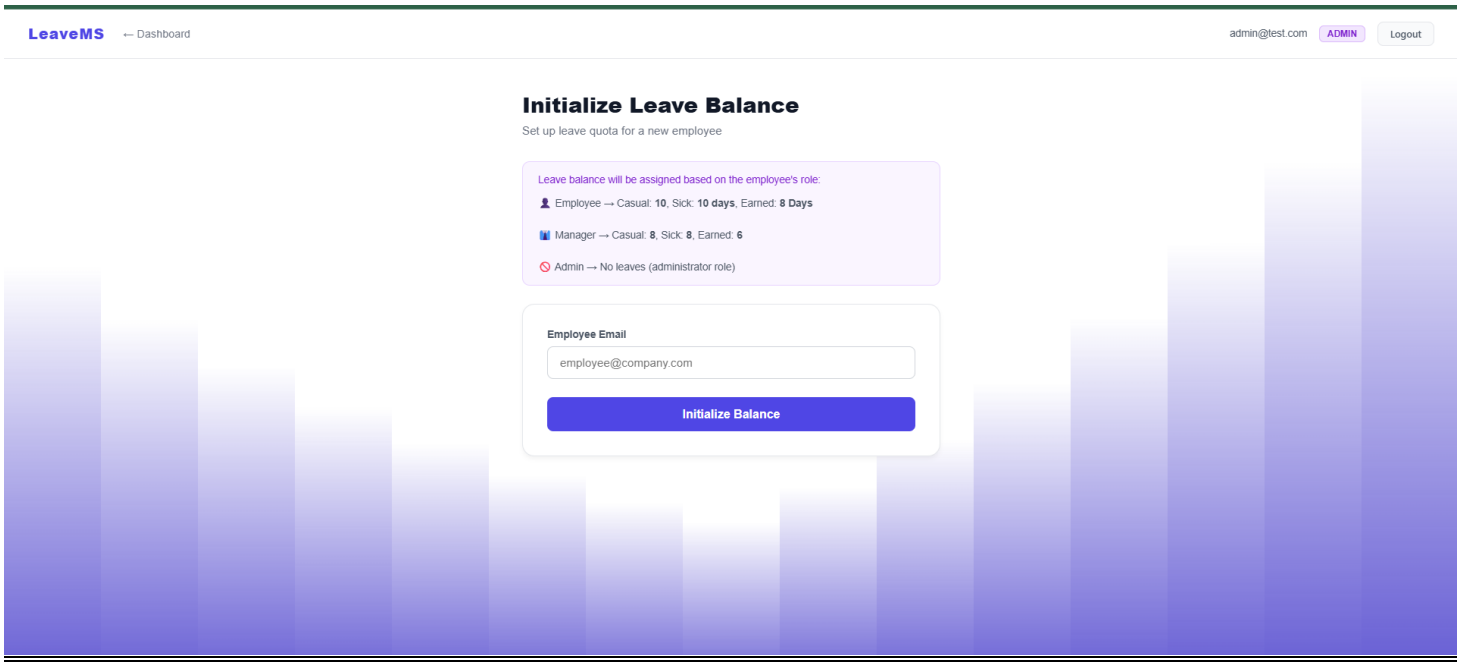


Figure 12.2.14 Employee Directory:

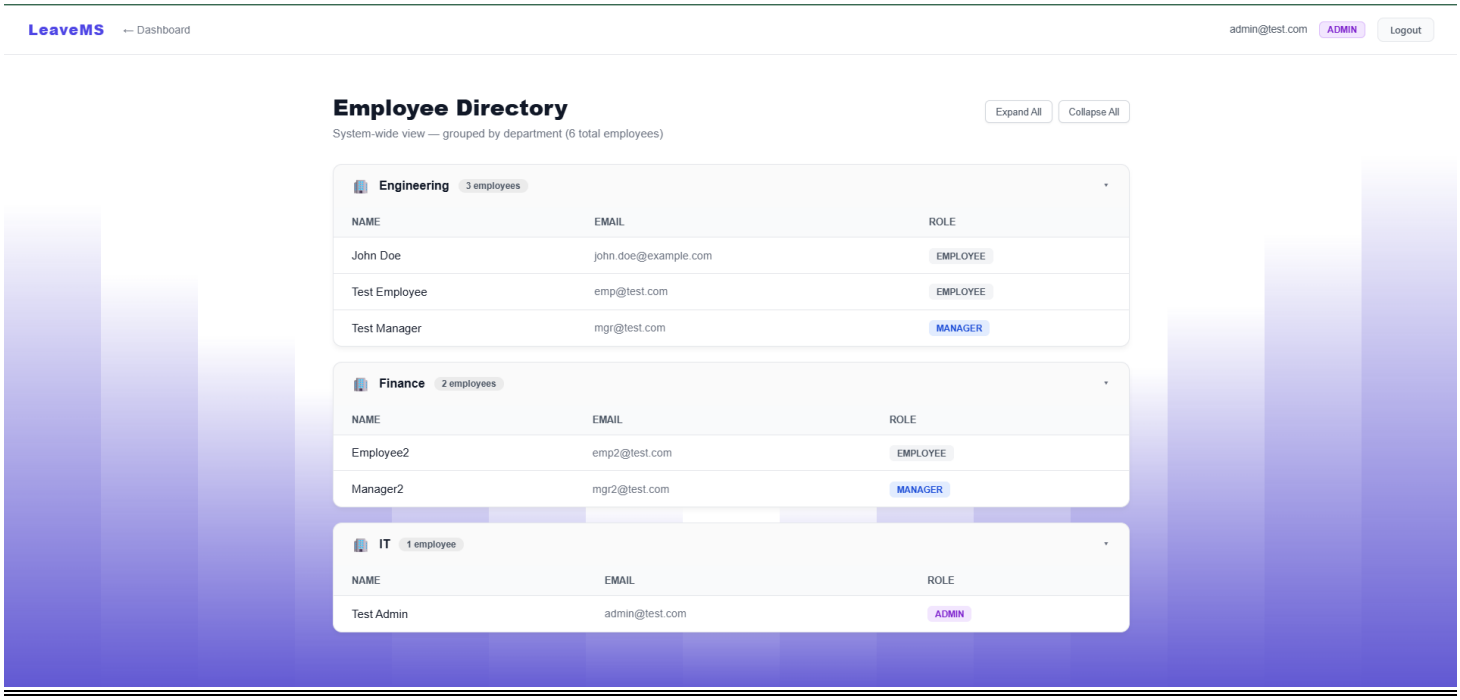
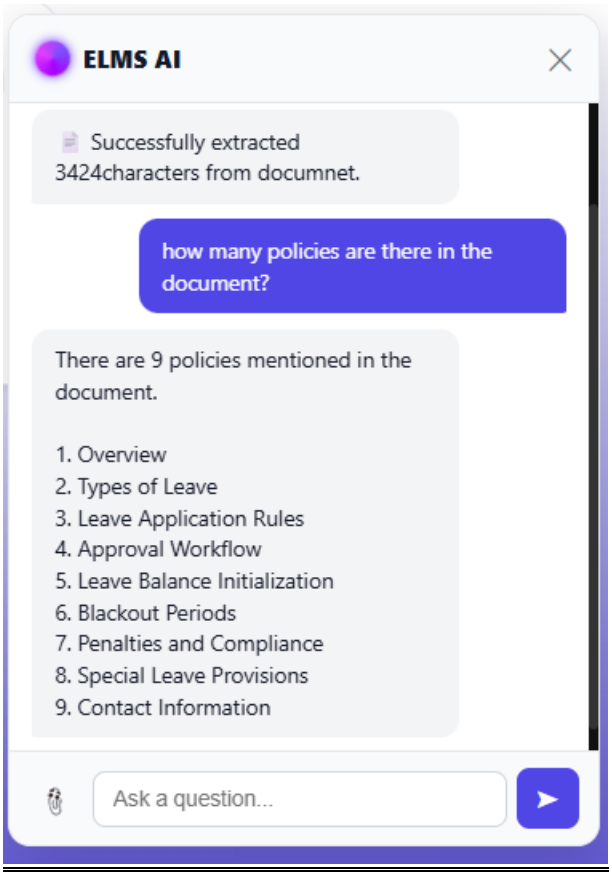


Figure 12.2.15 Admin's ChatBot PDF Upload and Response:



13. CONCLUSION

The **Employee Leave Management System (ELMS)** project was successfully designed, developed, and tested as a comprehensive enterprise web application. By shifting away from traditional monolithic designs, the application fulfills all objectives outlined in the problem statement through modern architectural paradigms:

1. **Microservices Scalability:** The separation of concerns into distinct Spring Boot services (Auth, User, Leave, AI) ensures the system is fault-tolerant and highly scalable. The integration of Eureka and an API Gateway establishes a robust networking foundation.
2. **Data Integrity:** The use of MySQL provides rigid relational structure, ensuring leave balances and hierarchical data remain consistent and ACID-compliant.
3. **Role-Based Accountability:** The three-tier RBAC model ensures clear ownership. The dynamic department-scoping allows managers to see exactly what they need to manage their specific teams efficiently.
4. **Intelligent Assistance:** The integration of the AI Chatbot utilizing Nvidia NIM and RAG methodology bridges the gap between static data and user experience. Employees can interact conversationally to understand policies or check statuses, drastically reducing the administrative burden on HR.
5. **Modern User Experience:** The React.js frontend delivers a responsive, light-themed UI with dynamic animated backgrounds, consumer-grade experience for enterprise software.

The project demonstrates the viability and immense value of combining robust backend microservices with cutting-edge Generative AI to modernize internal enterprise workflows.

14. SUMMARY

This project delivered a fully functional web application, **ELMS**, for Tata Technologies. Built using **React.js** for the frontend and **Spring Boot Microservices** backed by **MySQL** for the backend, the application solves the problem of fragmented leave tracking and tedious policy navigation.

Key accomplishments include:

- A **Distributed Backend** utilizing Netflix Eureka for service discovery and Spring Cloud Gateway for centralized API routing and security.
- **Secure Authentication** using JWTs, ensuring stateless and scalable session management across all microservices.
- **Relational Database Design** using Spring Data JPA and MySQL to handle complex entities (Users, Leaves, Balances) securely.
- **Role-Based Access Control** tailored for Employees, Managers, and Admins, featuring department-level data filtering for middle management.
- An **AI Chatbot** built on a custom Spring Boot service that utilizes RestTemplate for inter-service data gathering and Nvidia NIM for LLM inference, providing context-aware answers to users.
- A **Single Page Application** featuring clean aesthetics, robust form validation, and real-time state management.

Future enhancements identified include:

- Implementation of email/SMS notifications on ticket status changes.
- Advanced data visualization and analytics dashboards for Administrators.
- Integration with organizational LDAP/SSO for enterprise-grade identity management.

15. REFERENCES

Ref ID	Reference Details	Used in Pg#
1.	React.js Official Documentation, "Getting Started with React," Meta Platforms, Inc. Available: https://react.dev/	[Pg. 13 , (Sec 10.3 Frontend)]
2.	Spring Boot Reference Guide, "Building an Application with Spring Boot," VMware, Inc. Available: https://docs.spring.io/spring-boot/	[Pg. 13,14 (Sec 10.4 & 11.2 Architecture)]
3.	Spring Cloud Gateway Documentation, VMware, Inc. Available: https://docs.spring.io/spring-cloud-gateway/	[Pg. 13, 14 (Sec 10.4 & 11.2 API Gateway)]
4.	Netflix Eureka Documentation, "Service Discovery: Eureka," Spring Cloud Netflix. Available: https://docs.spring.io/spring-cloud-netflix/	[Pg. 13, 14 (Sec 10.4 & 11.2 Service Discovery)]
5.	MySQL Documentation, "MySQL 8.0 Reference Manual," Oracle Corporation. Available: https://dev.mysql.com/doc/refman/8.0/en/	[Pg. 13, 15 (Sec 10.5 & 11.3 DB Design)]
6.	JSON Web Tokens (JWT), "Introduction to JSON Web Tokens," Auth0. Available: https://jwt.io/introduction	[Pg. 13, 18 (Sec 10.6 & 11.7 Auth Flow)]
7.	NVIDIA API Catalog, "NVIDIA NIM Documentation and API Reference," NVIDIA Corporation. Available: https://build.nvidia.com/	[Pg. 13 (Sec 10.7 Generative AI RAG)]
8.	Framer Motion, "A production-ready motion library for React," Framer. Available: https://www.framer.com/motion/	[Pg. 13 (Sec 10.8 UI/UX Design)]
9.	Axios Documentation, "Promise based HTTP client for the browser." Available: https://axios-http.com/docs/intro	[Pg. 17 (Sec 11.6 Component Architecture)]

16. BIBLIOGRAPHY

1. Walls, C., "Spring in Action," 6th Edition, Manning Publications, 2022.
2. Newman, S., "Building Microservices: Designing Fine-Grained Systems," 2nd Edition, O'Reilly Media, 2021.
3. Flanagan, D., "JavaScript: The Definitive Guide," 7th Edition, O'Reilly Media, 2020.
4. Stefanov, S., "React: Up & Running," O'Reilly Media, 2nd Edition, 2022.
5. GeeksforGeeks, "ReactJS Tutorial," Available: https://www.geeksforgeeks.org/react/
6. Baeldung, "Spring Cloud Tutorial," Available: https://www.baeldung.com/spring-cloud-series
7. Pressman, R. S., "Software Engineering: A Practitioner's Approach," 9th Edition, McGraw-Hill, 2020.

17. ANNEXURE - INTERIM REPORT

1. Project Title: Employee Leave Management System (ELMS)
2. Intern Name: **SHREYASH JAISWAL**
3. Reporting Period: 1 Month of Internship
4. Project Overview: The objective of this project is to develop a highly scalable, role-based Employee Leave Management System for Tata Technologies. The system is designed to streamline leave applications, hierarchical approvals, and policy queries using a distributed Spring Boot Microservices architecture and an AI-powered conversational interface.
5. Work Completed During the Interim Period (Weeks 1-4):

- **Requirement Analysis & Technology Selection:** Finalized the tech stack (React.js for frontend, Spring Boot and MySQL for the backend). Conducted a literature survey of existing HR/Leave Management solutions.
- **System Architecture & Database Design:** Designed the client-server microservices architecture and drafted the relational database schema for user profiles, leave requests, and balances.
- **Backend Setup**: Configured the Spring Boot projects, set up the Netflix Eureka Discovery Server, configured the Spring Cloud API Gateway, and implemented secure JWT (JSON Web Token) authentication.
- **Initial Frontend Prototypes:** Scaffolded the React.js Single Page Application and created the initial UI layouts for the Login/Signup pages and the basic navigation structure.

6. Challenges Faced:

- Orchestrating inter-service communication (e.g., leave-service securely querying user-service) and maintaining JWT context across API Gateway routes.
- Designing the prompt engineering logic for the AI Chatbot to reliably retrieve real-time JSON database context and answer strictly based on organizational HR policies.